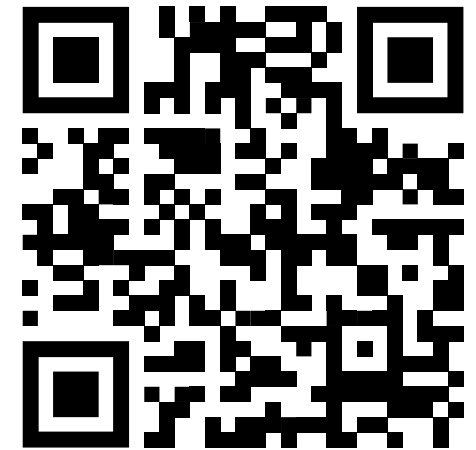


Programmierung in C++

Basic Objektorientierung

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Einführung

Lernziele

- Grundbegriffe der **Objektorientierung** sind bekannt
- Der **Sinn und Nutzen** von Klassen sind verstanden

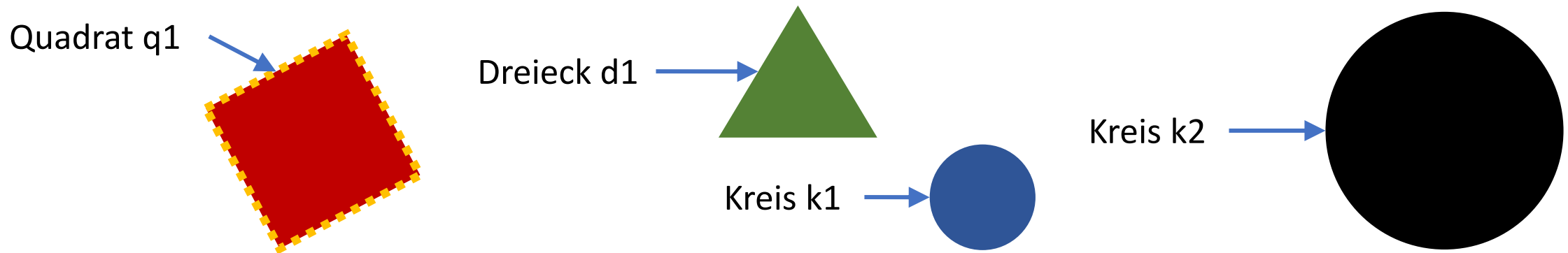
Einführung

Objekte, Attribute, Attributwerte, Methoden



Dargestellte Information wird in **Objekte** gegliedert. Jedes Objekt hat einen eindeutigen Namen, den **Objektnamen** oder **Bezeichner**. Die Merkmale der Objekte nennt man **Attribute**. Auch Attribute haben Bezeichner. Den Wert eines Attributs nennen wir **Attributwert**.

Bezeichner von Objekten und Attributen wollen wir mit kleinem Anfangsbuchstaben schreiben und ohne Leerzeichen. Besteht ein Bezeichner aus mehreren Worten, so schreiben wir nach jedem Wort groß („Camel Case“-Notation: *halloWelt*, *diesIstCamelCase*).



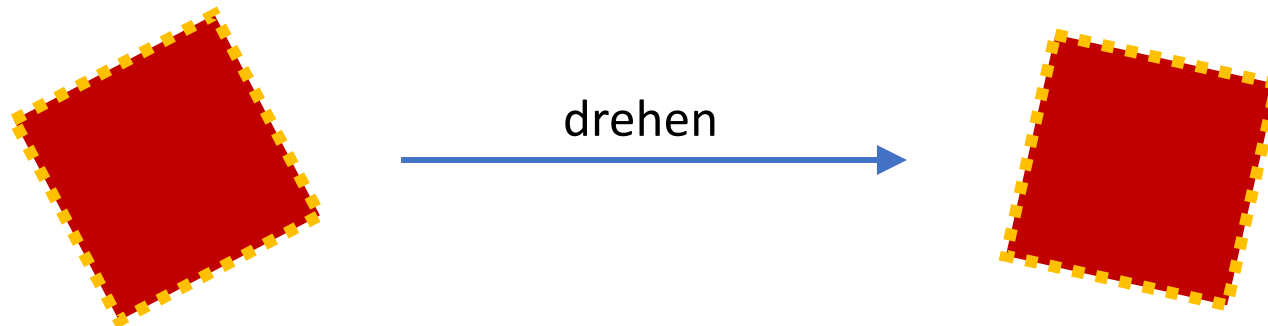
Einführung

Objekte, Attribute, Attributwerte, Methoden



Objekte besitzen bestimmte Fähigkeiten. Sie können festgelegte **Methoden** ausführen. Damit ein Objekt eine Methode ausführt, muss man ihm eine **Nachricht senden** (eine Botschaft senden), d.h. man muss die Methode bei dem Objekt **aufrufen**.

Methodennamen schreiben wir klein und ohne Leerzeichen. Nach jedem Wort schreiben wir groß.



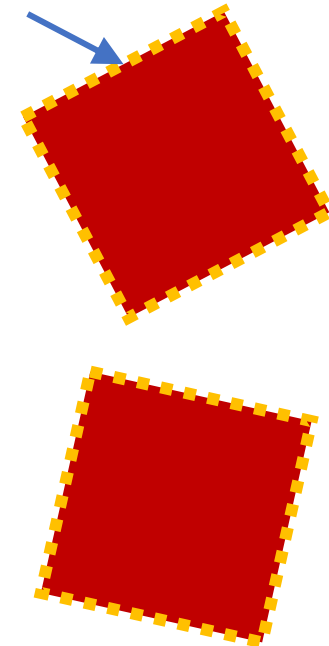
Einführung

Objekte, Attribute, Attributwerte, Methoden



- Attribute geben an, was ein Objekt „weiß“
 - Der Zugriff auf Attributwerte erfolgt über die **Punktnotation**
 - Attribute sind **Variablen**, die zu einem Objekt gehören
 - `q1.farbe = rot;`
- Methoden geben an, was ein Objekt „tut“
 - Der Aufruf einer Methode erfolgt ebenfalls über eine **Punktnotation**
 - Methoden sind **Funktionen**, die zu einem Objekt gehören
 - `q1.drehen();`
- Attribute und Methoden sind Teile eines Objekts, sogenannte **Member**
 - Anstelle von Attribute spricht man auch von **Member-Variablen**
 - Anstelle von Methoden spricht man auch von **Member-Funktionen**

Quadrat q1



Einführung

Klassen



Alle Objekte mit gleichen Attributen und gleichen Methoden werden durch eine **Klasse** beschrieben. Statt Klasse sagt man auch **Objekttyp**.

Bezeichner von Klassen schreiben wir groß und ohne Leerzeichen. Nach jedem Wort schreiben wir groß. Objekte einer Klasse bezeichnen wir auch als **Instanz** der Klasse.

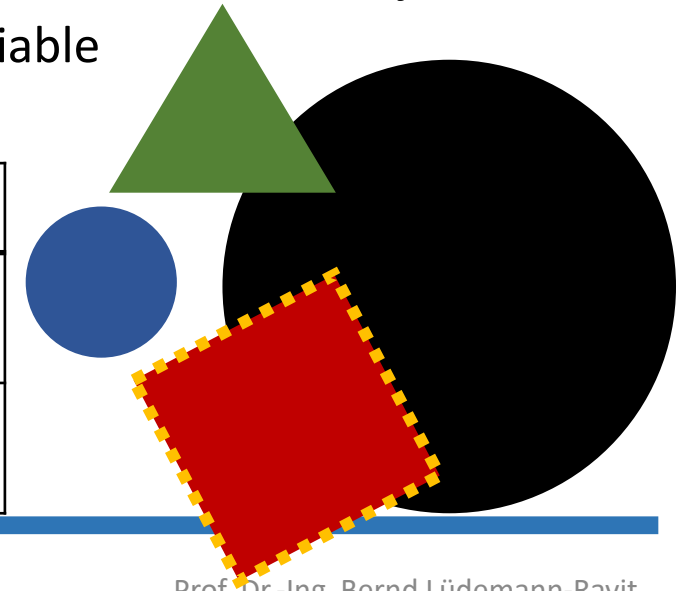
- Klassen bündeln die Gemeinsamkeiten unterschiedlicher Objekte
 - Sie fassen zusammen, was diese Objekte ausmacht, sind aber selbst noch kein Objekt
 - Mit der Doppelpunktnotation wird eine Funktion oder eine Variable als Member der Klasse gekennzeichnet
 - `Form::drehen()`

Klassenname

Attribute

Methoden

Form
farbe
rahmen
drehen() flaecheninhalt()



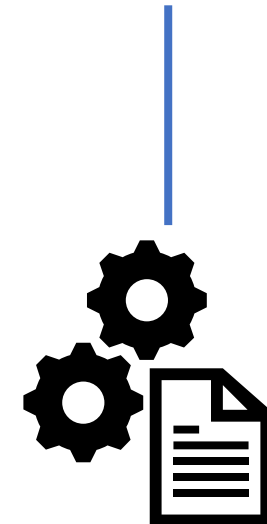
Einführung

Klassen – Trennen von Deklaration und Definition

- Klassen in C++ sind in zwei **Dateien** aufgeteilt.
- Die **Header-Datei** (.h):
 - Hier stehen die Member der Klasse mit ihren Eigenschaften aufgelistet -> Die Deklarationen der Member
 - Sie beschreibt, was die Klasse „besitzt“
 - Dient als Schnittstelle für andere Dateien, damit sie wissen, wie sie Objekte der Klasse verwenden können
- Die **Quellcode-Datei** (.cpp):
 - Hier befindet sich die Funktionalität der Klasse -> Die Definitionen der Member
 - Enthält die tatsächliche Implementierung der Methoden
 - Setzt die Beschreibung aus der Header-Datei um
 - Ist für andere Klassen / Dateien nicht sichtbar



Form
farbe rahmen
drehen() flaecheninhalt()

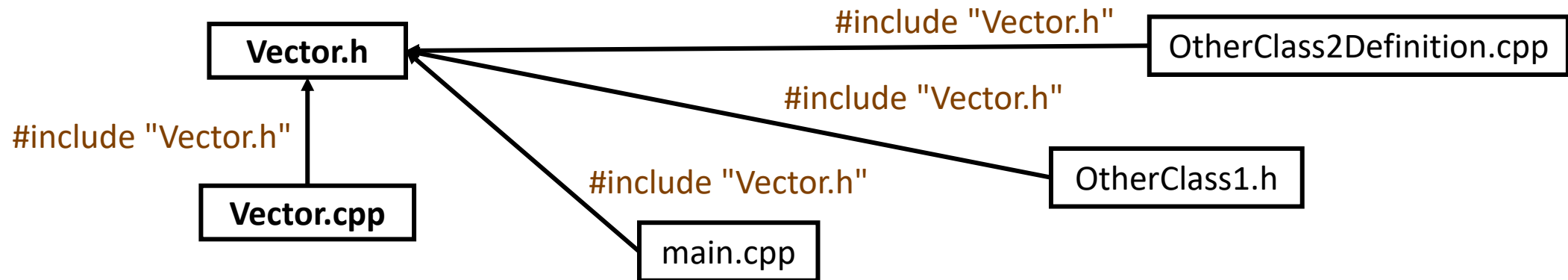


Einführung



Klassen – Trennen von Deklaration und Definition

- Wir trennen bei Klassen die Definition von der Deklaration
 - Dadurch bleibt die Klasse übersichtlich und ist leichter anpassbar.
 - Außerdem können wir die Klasse dadurch leichter in andere Dateien mit einbinden, indem wir einfach die Header-Datei inkludieren.
 - Würden wir die Funktionsdefinition in andere Dateien mit einbinden, müsste für jede dieser Dateien bei einer Änderung der Implementierung die komplette Datei der Funktionsdefinition neu kompiliert werden.



Einführung

Geltungsbereiche

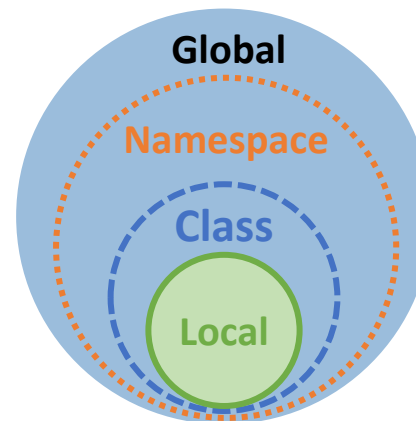


Local Scope

- Eine Variable, die in einer Funktion deklariert wird, heißt **lokale Variable**.
 - Ihr Geltungsbereich erstreckt sich vom Punkt der Deklaration bis zum Ende des Blocks
 - Ein *Block* wird durch ein Paar von geschweiften Klammern eingegrenzt
- Funktionsargumente gelten als lokale Variablen

Class Scope

- Eine Variable heißt **Member-Variable**, wenn sie in einer Klasse deklariert wird.
 - Außer sie wird innerhalb einer Klasse in einer Funktion oder Enumeration deklariert
- Ihr Geltungsbereich erstreckt sich über den gesamten Block, in der sie deklariert ist



Namespace Scope

- Ist eine Variable außerhalb von Klassen, Funktionen und Enumerationen deklariert, so heißt sie ...
- ... **Namespace Member-Variable**, wenn sie in einem Namespace deklariert ist.
 - Ihr Geltungsbereich erstreckt sich vom Punkt der Deklaration bis zum Ende des Namespace
 - Mehr dazu in einer späteren VL
- ... **Globale Variable**, wenn sie außerhalb eines Namespace deklariert ist.
 - Ihr Geltungsbereich erstreckt sich über das gesamte Programm

Einführung

Zusammenfassung



- **Basis der Objektorientierten Programmierung in C++**
- **Klassen** verbinden **Daten/Attribute** und **Operationen/Methoden** von Objekten/Instanzen
- Klassen repräsentieren **benutzerdefinierte Datentypen / Entitäten**
- Klassen ermöglichen die **Trennung** zwischen der Anwendung (Schnittstellen/Header-Datei) einer Entität und seiner tatsächlichen Implementierung (cpp-Datei)
 - Der **Zugriff** auf die Klasse erfolgt über frei verwendbare Funktionalitäten (Variablen, Methoden)
 - Die klassenspezifische Implementierung besitzt Zugriff auf Variablen, die ansonsten **nicht zugänglich** sind
 - Die Trennung soll eine **einfache und konsistente Nutzung durch eine definierte Schnittstelle** der Entität sowie die stetige **Optimierung der Implementierung** gewährleisten
- Gut gebaute und benannte Klassen helfen beim **Verständnis** und der Implementierung
 - **Attribute** der Klasse geben an, was Objekte der Klasse „wissen“
 - **Methoden** der Klasse geben an, was Objekte der Klasse „tun“

Einführung

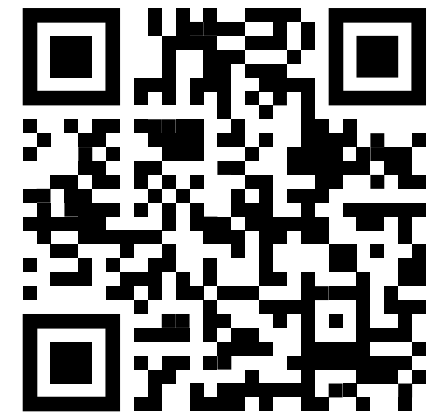
Verständnisfragen



■ WAHR oder FALSCH?

- Variablen, die zu einer Klasse gehören, nennt man Attribute.
- Methoden sind die Funktionen einer Klasse.
- Die Header-Datei einer Klasse inkludiert die eigene Quellcode-Datei.
- Alle Instanzen einer Klasse greifen auf denselben Speicherbereich zu.
- Klassen bündeln die Gemeinsamkeiten unterschiedlicher Objekte.

PIN: 569316



<https://poll.hs-kempten.de/poll/>



Aufbau

Lernziele

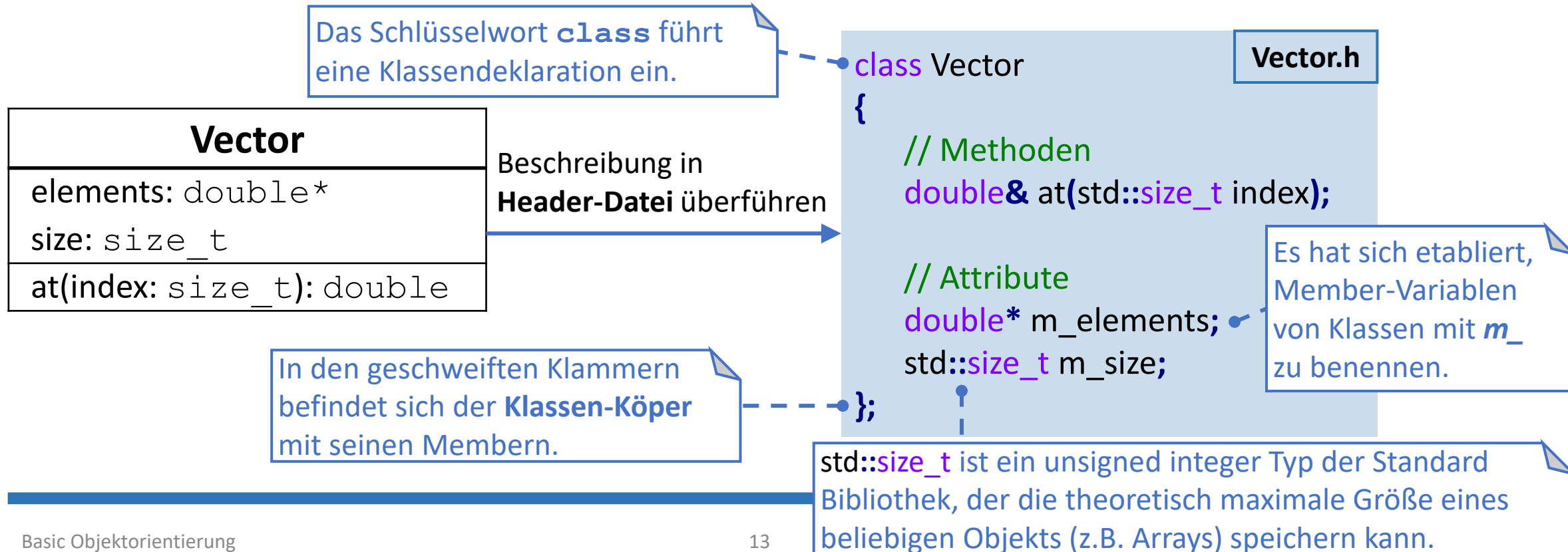
- Es können eigene **Klassen erstellt** und mit **Funktionalitäten versehen** werden
 - Dazu wird in diesem Kapitel die Klasse „Vector“ beispielhaft Schritt für Schritt aufgebaut.
 - Die Klasse „Vector“ soll ein Array mit beliebig vielen Elementen verwalten können und Funktionen auf den Elementen bereitstellen.

Aufbau

Eine Vector-Klasse



- Wir wollen die Klasse **Vector** erstellen
 - Sie soll ein Array von Double-Werten verwalten
 - Von außerhalb der Klasse soll auch auf die einzelnen Elemente zugegriffen werden können
 - Innerhalb des Klassenkörpers wird auf die Doppelpunktnotation verzichtet

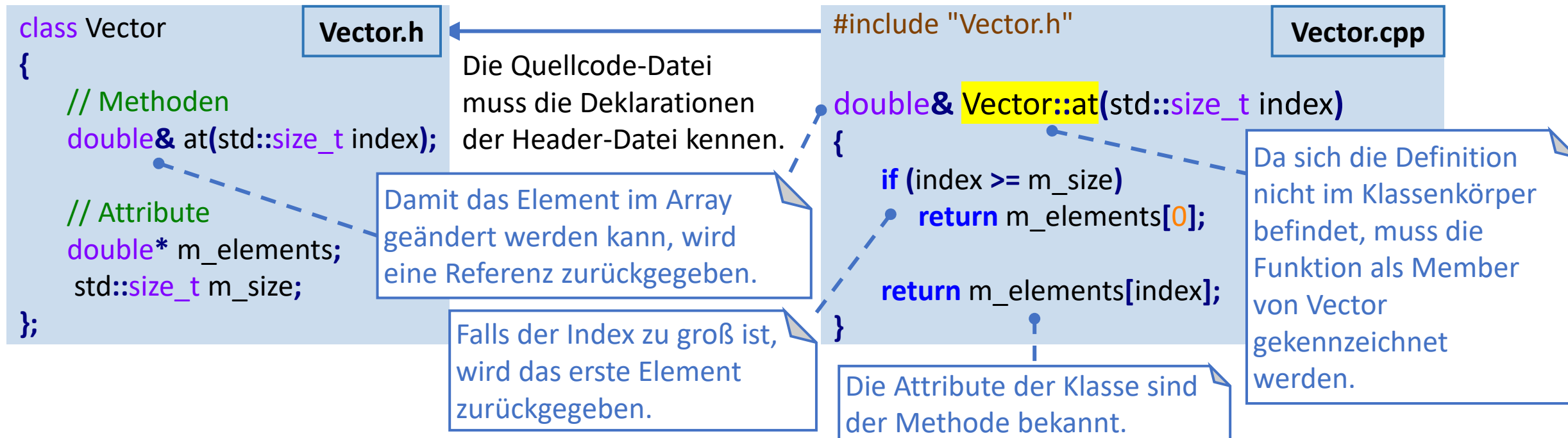


Aufbau

Methoden



- Methoden sind Funktionen, die einer Klasse gehören
 - Methoden können wie Funktionen beliebige Parameter besitzen und einen Wert zurückgeben oder nicht
 - Methoden können auf die Attribute der eigenen Instanz zugreifen



Aufbau



Public Member

- Klassen bieten durch ihre **public** Member eine Schnittstelle zwischen internen Abläufen der Klasse und Zugriffen von außerhalb der Klasse
 - Auf public Member kann auch von außerhalb der Klasse zugegriffen werden
 - Zugriff auf Member mit dem . Operator (Bei Zeigern auf Instanzen mit dem -> Operator)

```
class Vector {  
public: // Die nachfolgenden Member sind public  
    double& at(std::size_t index);  
  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

Eine Klasse, bei der alle Member *public* sind, fungiert ähnlich wie ein *Struct*.

Vector v1:
elements:

--

size:

6

 →

0:	1:	2:	3:	4:	5:
1.2	2.3	3.4	4.5	5.6	6.7

```
#include "Vector.h"  
int main()  
{  
    // Initialisiert einen "Vector" mit 6 Elementen  
    Vector v1{  
        new double[6]{1.2,2.3,3.4,4.5,5.6,6.7},  
        6  
    };  
    // Gib den allokierten Speicherplatz wieder frei  
    delete[] v1.m_elements;  
}
```

Die Header-Datei muss bei Anwendung anderen Dokumenten zur Verfügung gestellt werden.

Die *public* Member-Variablen können wie bei einem *Struct* initialisiert werden.

Aufbau

Private Member



- **private** Klassen-Member sind nur innerhalb der Klasse zugänglich
 - Verhindert unerwünschte und unkontrollierte Zugriff von außerhalb der Klasse
 - Member, die keiner Zugriffs-Spezifikation untergeordnet sind, sind standardmäßig private

```
class Vector {  
public: // Die nachfolgenden Member sind public  
    double& at(std::size_t index);  
  
private: // Die nachfolgenden Member sind private  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

Auch die Initialisierung wie bei einem Struct funktioniert nun nicht mehr.

```
#include "Vector.h"  
int main()  
{  
    // Erstellt einen Vector v1  
    Vector v1;  
    v1.m_size = 5;  
}
```

ERROR: Auf Member "Vector::m_size" kann nicht zugegriffen werden.

Aufbau

Getter & Setter



- „**Getter**“ und „**Setter**“ sind public Schnittstellen-Methoden
 - Gewährleisten einen gesicherten und geregelten Zugriff auf ein privates Attribut
 - Setter können sicherstellen, dass einem Attribut immer ein sinnvoller Wert zugewiesen wird
 - Getter verändern das Attribut nicht und ermöglichen auch keine Änderung von außerhalb der Klasse

```
class Vector {  
public:  
    void setSize(std::size_t size); // Setter  
    std::size_t getSize(); // Getter  
  
    double& at(std::size_t index);  
private:  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

```
#include "Vector.h"  
void Vector::setSize(std::size_t size) {  
    m_size = size;  
};  
  
std::size_t Vector::getSize() {  
    return m_size;  
};  
  
double& Vector::at(std::size_t index) { /*...*/ }
```

Vector.cpp

Aufbau



Konstruktor – Motivation

- Bei Änderung von *m_size* muss entsprechend Speicherplatz in der Größe reserviert werden
 - **Frage:** Sollte hierzu einfach neuer Speicherplatz auf dem *m_elements* Pointer allokiert werden?

```
void Vector::setSize(std::size_t size)
{
    m_size = size;
    m_elements = new double[m_size];
};
```

```
int main() {
    Vector v1;
    while (true)
        v1.setSize(50);
}
```

Aufbau



Konstruktor – Motivation

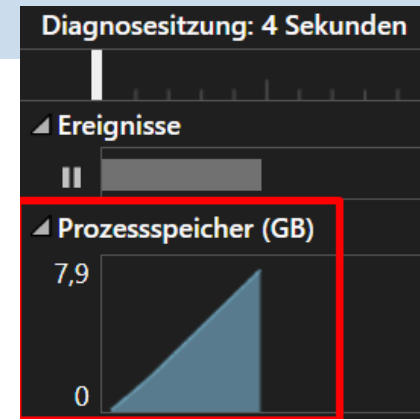
- Bei Änderung von *m_size* muss entsprechend Speicherplatz in der Größe reserviert werden
 - **Frage:** Sollte hierzu einfach neuer Speicherplatz auf dem *m_elements* Pointer allokiert werden?

```
void Vector::setSize(std::size_t size)
{
    m_size = size;
    m_elements = new double[m_size];
};
```

```
int main() {
    Vector v1;
    while (true)
        v1.setSize(50);
}
```



- Da der zuvor allokierte Speicherplatz nicht freigegeben wird, entstehen Memory Leaks
- Der Speicherplatz wird gefüllt und kann vom Programm nicht mehr aufgeräumt werden



Aufbau



Konstruktor – Motivation

- Bei Änderung von *m_size* muss entsprechend Speicherplatz in der Größe reserviert werden
 - Der vorher allokierte Speicherplatz von *m_elements* muss zuvor freigegeben werden
 - **Frage:** Was passiert, wenn wir vor der neuen Allokation den Speicher freigeben?

```
void Vector::setSize(std::size_t size) {  
    m_size = size;  
    delete[] m_elements;  
    m_elements = new double[m_size];  
};
```

```
int main() {  
    Vector v1;  
    while (true)  
        v1.setSize(50);  
}
```

Aufbau



Konstruktor – Motivation

- Bei Änderung von *m_size* muss entsprechend Speicherplatz in der Größe reserviert werden
 - Der vorher allokierte Speicherplatz von *m_elements* muss zuvor freigegeben werden
 - **Frage:** Was passiert, wenn wir vor der neuen Allokation den Speicher freigeben?

```
void Vector::setSize(std::size_t size) {  
    m_size = size;  
    delete[] m_elements;  
    m_elements = new double[m_size];  
};
```

```
int main() {  
    Vector v1;  
    while (true)  
        v1.setSize(50);  
}
```

- Das Programm stürzt zur Laufzeit ab, da beim Aufruf von *setSize* noch kein gültiger Wert für *m_elements* gesetzt wurde
- Zur Lösung dieses Problems bieten sich **Konstrukturen** an!



Ausgelöste Ausnahme

Ausnahme ausgelöst bei 0x00007FFF60F6499E (ucrtbased.dll) in
P2Sandbox.exe: 0xC0000005: Zugriffsverletzung beim Lesen an
Position 0xFFFFFFFFFFFFFFFF.

Aufbau

Konstruktor



```
class Vector {  
public:  
    // Standard-Konstruktor  
    Vector();  
  
    void setSize(std::size_t size);  
    std::size_t getSize();  
    double& at(std::size_t index);  
  
private:  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

Der Doppelpunkt kennzeichnet die Initialisierung der Member-Variablen.

Standard-Konstruktor für die Klasse Vektor.

```
#include "Vector.h"  
Vector::Vector()  
    : m_elements{nullptr}  
    , m_size{0}  
{  
}  
/* ... */
```

Vector.cpp

Setze das Array explizit auf `nullptr`, um zu signalisieren, dass kein Speicher allokiert ist.

```
int main() {  
    Vector v1;  
    while (true)  
        v1.setSize(50);  
}
```

Der Standard-Konstruktor wird aufgerufen und der Speicher wird allokiert.

Die vorherigen Probleme treten NICHT mehr auf!

Aufbau

Standard-Konstruktor & Attribut-Initialisierung



- Eine Funktion mit demselben Namen wie die Klasse heißt **Konstruktor**
 - Der Konstruktor mit leerer Parameterliste heißt **Standard-Konstruktor** und wird vom System automatisch angelegt, wenn kein eigener Konstruktor definiert ist
 - Ein Konstruktor wird zur Initialisierung der Objekte einer Klasse verwendet
- Die **Initialisierung der Attribute** findet in der Definition des Konstruktors vor seinem Funktionskörper statt
 - Variablen, denen erst im Funktionskörper ein Wert zugewiesen wird, sind zum Lebensbeginn eines Objekts nicht initialisiert
 - Ein Doppelpunkt nach dem Funktionskopf kennzeichnet die **Initialisierung** der Member-Variablen
 - Konstante Member-Variablen können nur an dieser Stelle initialisiert werden

```
class Vector {  
public:  
    Vector();  
    /* ... */  
}
```

Vector.h

```
#include "Vector.h"  
Vector::Vector()  
    : m_elements{nullptr}  
    , m_size{0}  
{  
}  
/* ... */
```

Vector.cpp

Best Practice: An dieser Stelle alle Attribute explizit mit einem Wert initialisieren.

Aufbau

Konstruktor



```
class Vector {  
public:  
    Vector(std::size_t size);  
  
    void setSize(std::size_t size);  
    std::size_t getSize();  
    double& at(std::size_t index);  
private:  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

- Durch die Angabe eines Konstruktors mit Parametern (`size`) wird der Standard-Konstruktor nicht mehr implizit erzeugt

Dem Konstruktor muss die Größe direkt übergeben werden.

```
#include "Vector.h"  
Vector::Vector(std::size_t size)  
    : m_elements{new double[size]}  
    , m_size {size}  
{  
    for (std::size_t i=0; i < m_size; ++i)  
        m_elements[i] = 0;  
}  
/* ... */
```

Vector.cpp

Der Standard-Konstruktor kann nicht mehr aufgerufen werden.

Initialisiere die Attribute anhand des übergebenen Parameters.

```
// Akzeptiert nur einen Parameter vom Typ 'Vector'  
void printSize(Vector v) {  
    std::cout << "Size: " << v.getSize() << '\n';  
}
```

```
int main() {
```

```
// Initialisierung ohne gegebene Größe nicht möglich  
Vector v0; // FEHLER: kein Standardkonstruktor vorhanden.
```

```
// Initialisiert einen Vector mit 6 Elementen
```

```
Vector v1{ 6 };  
printSize(v1);
```

Size: 6

```
// Der Konstruktor kann implizit aufgerufen werden  
printSize('a');
```

Size: 97

Aufbau

Konstruktor – `explicit` Schlüsselwort



```
class Vector {  
public:  
    Vector(std::size_t size);  
    /* ... */  
}
```

Vector.h

```
void printSize(Vector v)  
{    std::cout << "Size: " << v.getSize() << '\n'; }  
int main() {  
    printSize('a'); // Impliziter Aufruf des Konstruktors  
}
```

- Das Schlüsselwort **`explicit`** verhindert, dass ein Konstruktor durch die Angabe von passenden Parametern implizit aufgerufen werden kann.
 - Zwingt den Anwender dazu, Instanzen der Klasse explizit zu erzeugen
 - Verhindert unerwartetes und unerwünschtes Verhalten

```
class Vector {  
public:  
    explicit Vector(std::size_t size);  
    /* ... */  
}
```

Vector.h

```
void printSize(Vector v)  
{    std::cout << "Size: " << v.getSize() << '\n'; }  
int main() {  
    printSize('a'); // FEHLER!  
}
```

Aufbau

Destruktor



- Der **Destruktor** ist das Komplement zum Konstruktor
 - Wird durch das Komplementsymbol `~` gefolgt vom Namen der Klasse deklariert
 - Er wird automatisch/implizit am Ende der Lebenszeit eines Objekts aufgerufen
 - Speicherplatz, der durch das Objekt allokiert wurde, muss hier wieder freigegeben werden
- Durch die Konstruktoren und den Destruktor muss sich der Nutzer einer Klasse nicht um dessen Implementierung Gedanken machen
 - Der Nutzer muss sich nicht selbst um die Allokation und Deallokation des Speichers für Daten des Objekts kümmern

```
class Vector {  
public:  
    // ...  
    ~Vector();  
    // ...  
};
```

Vector.h

```
#include "Vector.h"  
Vector::~~Vector() {  
    delete[] m_elements;  
}
```

Vector.cpp

```
int main() {  
    Vector v1{6};  
    Vector* ptr{ new Vector{4} };  
    delete ptr;  
    if (!ptr) {  
        Vector v2{4};  
    }  
}
```

Destruktor-Aufruf für *ptr

Destruktor-Aufruf für v2

Destruktor-Aufruf für v1

Aufbau

Zusammenfassung



- Klassen sind wesentlich, um **eigene Datentypen** zu definieren, die **einfach und intuitiv bedienbar** sind, ohne dass die intern gespeicherten Strukturen und vorhandenen Umsetzungen für einen Anwender bekannt sein müssen.
 - Der Entwickler muss sich Gedanken machen, was einem Anwender der Klasse zur Verfügung gestellt wird (**public**) und was nicht (**private**).
 - **Getter** und **Setter** gewährleisten einen korrekten und intuitiven Zugriff auf die Member-Variablen der Klasse.
- Der **Konstruktor** wird bei der Erzeugung einer Instanz aufgerufen
 - Er „baut“ das Objekt, indem er alle Member-Variablen initialisiert und nutzerdefinierten Code ausführt
 - Gibt der Nutzer keinen eigenen Konstruktor an, wird ein **Standard-Konstruktor** erzeugt
- Der **Destruktor** wird bei der Vernichtung einer Instanz aufgerufen
 - Er „räumt“ das Objekt wieder „auf“

Aufbau

Verständnisfragen



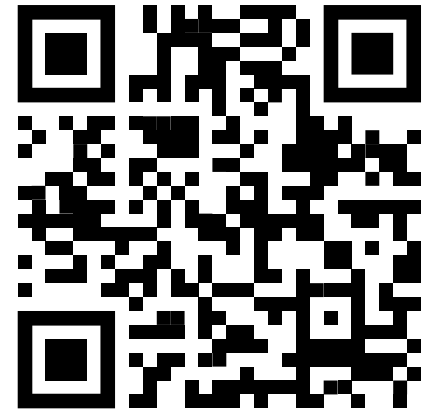
■ Was ist ein Konstruktor einer Klasse?

- Eine Methode zum Ändern des internen Zustands der Klasse.
- Eine Methode zum Initialisieren von Membern, wenn ein Objekt erstellt wird.
- Eine Methode zum Löschen eines Objekts, wenn es außerhalb des Gültigkeitsbereichs fällt.
- Eine Methode zum Zugriff auf private Member von außen.

■ WAHR oder FALSCH?

- Ein mit **explicit** deklarierter Konstruktor verhindert die implizite Typumwandlung der Parameter des Konstruktor-Aufrufs.
- Soll eine Klasseninstanz sowohl durch Angabe von Parametern als auch mit einer leeren Parameterliste konstruiert werden können, muss der Standard-Konstruktor explizit implementiert werden.
- Ein Konstruktor gewährleistet, dass jede Member-Variable mit einem Standard-Wert initialisiert wird.
- Destruktoren geben den gesamten Speicherplatz frei, der zur Laufzeit von einer Instanz der Klasse allokiert wurde.

PIN: 569316



<https://poll.hs-kempten.de/poll/>



Anwendung

Lernziele

- Instanzen können erstellt und verwendet werden
 - Es ist klar, in welchem Scope sich die Instanzen befinden
 - Der Unterschied zwischen Initialisierung und Zuweisung ist deutlich geworden
 - Mit Pointern und Referenzen auf Instanzen kann umgegangen werden
 - Der `this`-Pointer ist bekannt und wurde verstanden
- Zur Dokumentation von Klassen können geeignete Kommentare erzeugt werden

Anwendung

Initialisierung



- Beim Erzeugen von Objekten zwischen `()` und `{ }` unterscheiden
 - Um Initialisierung und Zuweisung zu unterscheiden, sollte auf eine Initialisierung mit `=` verzichtet werden
 - `{ }` kann verwendet werden, um Attribute mit einem Default-Wert zu initialisieren
 - Da `()` bei Funktionen verwendet werden, kann der Compiler eine Instanziierung mit einer Funktionsdefinition verwechseln

```
// ruft den Standard-Konstruktor auf:  
Widget w1;  
// Keine Zuweisung; ruft den Kopier-Konstruktor:  
Widget w2 = w1;  
// Ist eine Zuweisung; ruft Kopier-Zuweisung:  
w1 = w2;
```

```
class Widget {  
    /* ... */  
private:  
    int x{ 0 }; // OK! x's Default-Wert ist 0  
    int y = 0;  // auch OK!  
    int z(0);   // FEHLER!  
};
```

```
Widget w1(10); // ruft Widget Konstruktor mit Argument 10  
Widget w2();   // Deklariert eine Funktion namens w2, dass ein Widget zurück gibt  
Widget w3{};   // ruft Widget Standard-Konstruktor
```

Anwendung



Initialisierung mit `std::initializer_list`

- Mit der **Initializer-List** aus der Standard Bibliothek können Objekte mit einer Liste von Elementen initialisiert werden
 - Mehr zu der `std::initializer_list` in einer späteren Vorlesung
 - Wird bei der Initialisierung mit `{ }` wann immer möglich aufgerufen

Ohne Initializer-List

```
class Vector {  
public:  
    Vector(std::size_t size);  
    Vector(std::size_t size, double defaultValue);  
  
    /* ... */  
};
```

```
Vector(10);    // ruft ersten ctor auf  
Vector{10};    // ruft auch ersten ctor auf
```

```
Vector(2, 5.0); // ruft zweiten ctor auf  
Vector{2, 5.0}; // ruft auch zweiten ctor auf
```

Mit Initializer-List

```
class Vector {  
public:  
    Vector(std::size_t size);  
    Vector(std::size_t size, double defaultValue);  
    Vector(std::initializer_list<double> ilist);  
    /* ... */  
};
```

```
Vector(10);    // ruft ersten ctor auf  
Vector{10};    // ruft initializer_list ctor auf  
                // erzeugt Vector mit einem Element 10.0  
Vector(2, 5.0); // ruft zweiten ctor auf  
Vector{2, 5.0}; // ruft initializer_list ctor auf  
                // erzeugt Vector mit Elementen 2.0 und 5.0
```

Anwendung

Initialisierung & Zuweisung



```
int main() {  
    // Initialisierung: Wir erstellen ein Objekt mit dem Konstruktor  
    Vector vec1(5); // Initialisiert einen Vektor mit 5 Elementen  
    std::cout << "vec1 Größe nach Initialisierung: " << vec1.getSize() << '\n';  
    // Die Werte des Vektors ansehen  
    for (std::size_t i = 0; i < vec1.getSize(); ++i) {  
        std::cout << "vec1[" << i << "] = " << vec1.at(i) << '\n';  
    }  
    // Zuweisung: Ein bestehendes Objekt wird verändert  
    vec1.setSize(3); // Die Größe von vec1 ändern  
    std::cout << "vec1 Größe nach Zuweisung: " << vec1.getSize() << '\n';  
    // Neue Werte zuweisen  
    for (std::size_t i = 0; i < vec1.getSize(); ++i) {  
        vec1.at(i) = i + 1; // 1, 2, 3  
    }  
    // Werte nach der Änderung ansehen  
    for (std::size_t i = 0; i < vec1.getSize(); ++i) {  
        std::cout << "vec1[" << i << "] = " << vec1.at(i) << '\n';  
    }  
}
```

```
class Vector {  
public:  
    Vector();  
    explicit Vector(std::size_t size);  
    ~Vector();  
    void setSize(std::size_t size);  
    std::size_t getSize();  
    double& at(std::size_t index);  
private:  
    double* m_elements;  
    std::size_t m_size;  
};
```

Vector.h

```
vec1 Größe nach Initialisierung: 5  
vec1[0] = 0  
vec1[1] = 0  
vec1[2] = 0  
vec1[3] = 0  
vec1[4] = 0  
vec1 Größe nach Zuweisung: 3  
vec1[0] = 1  
vec1[1] = 2  
vec1[2] = 3
```


Anwendung

Initialisierung & Zuweisung – Vorsicht bei Zuweisungen



```
int main() {  
    // Initialisierung: Wir erstellen ein Objekt mit dem Standard-Konstruktor  
    Vector v;  
    std::cout << "v Größe: " << v.getSize() << '\n'; v Größe: 0  
  
    // Zuweisung: Wir erstellen ein unbenanntes Objekt und weisen es v1 zu!  
    v = Vector(5);  
    std::cout << "v Größe: " << v.getSize() << '\n'; v Größe: 5  
}
```

Hier findet eine
Zuweisung statt!

Hier findet eine
Initialisierung statt!

- In der markierten Zeile wird zuerst eine unbenannte Vector-Instanz **instanziiert** und diese dann dem bereits instanziierten Vector v **zugewiesen**.
 - Nach der Zuweisung wird das unbenannte Objekt gelöscht => **Destruktor-Aufruf**
 - Die Zuweisung ist damit ineffizienter als eine direkte Instanziierung und birgt außerdem **Gefahren**
 - Durch den Destruktor-Aufruf wird der allokierte Speicherplatz direkt wieder freigegeben und der `m_elements`-Zeiger von v zeigt auf einen **nicht reservierten Speicherbereich!**



Anwendung

Initialisierung & Zuweisung – Vorsicht bei Zuweisungen

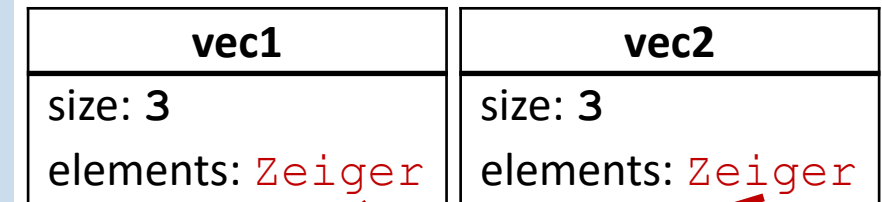


```
int main() {  
    /* ... */  
    // Initialisierung: Wir erstellen ein Objekt mit dem Standard-Konstruktor  
    Vector vec2;  
    std::cout << "vec2 Größe: " << vec2.getSize() << '\n';  
    // Zuweisung: v2 wird der Vector v1 zugewiesen => Es geschieht eine Kopie  
    vec2 = vec1;  
    std::cout << "vec2 Größe: " << vec2.getSize() << '\n';  
  
    // Werte nach der Änderung ansehen  
    for (std::size_t i = 0; i < vec1.getSize(); ++i) {  
        std::cout << "vec2[" << i << "] = " << vec2.at(i) << '\n';  
    }  
  
    // Wir ändern die Größe von vec2 auf 1  
    vec2.setSize(1);  
    // Ausgabe von vec1  
    for (std::size_t i = 0; i < vec1.getSize(); ++i) {  
        std::cout << "vec1[" << i << "] = " << vec1.at(i) << '\n';  
    }  
}
```

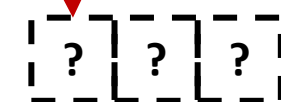
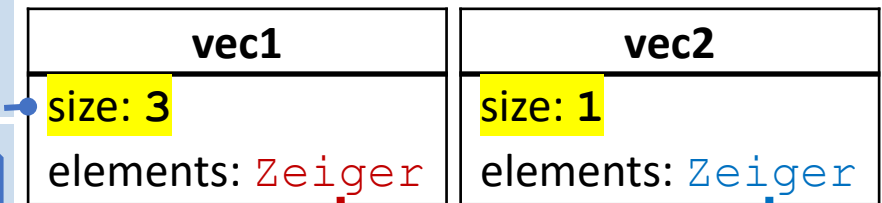
Die Größe und der Zeiger von vec1 werden **kopiert**. vec1 und vec2 **verwalten denselben Speicherplatz!**

Das Array von vec1 wurde gelöscht, aber die Instanz wurde darüber nicht informiert!

Mit dem Aufruf des Destruktors von vec1 **stürzt das Programm ab**, da der Speicherplatz von `elements` bereits freigegeben wurde!



```
vec2 Größe: 0  
vec2 Größe: 3  
vec2[0] = 1  
vec2[1] = 2  
vec2[2] = 3
```



```
vec1[0] = -1.45682e+144  
vec1[1] = -1.45682e+144  
vec1[2] = -1.45682e+144
```

Speichermüll

Anwendung

Zeiger, Arrays, Referenzen



- Zeiger und Referenzen auf Instanzen sowie Arrays von Instanzen können genauso wie bei Standard-Datentypen verwendet werden
- Die automatische Typdeduktion funktioniert auch bei Klassen

```
Vector* p{nullptr}; // Nicht-zugewiesener Zeiger
```

```
Vector v{5};  
p = &v; // Adressierung der Instanz 'v'  
  
p->setSize(3);  
std::cout << "Größen von *p: " << p->getSize()  
    << ", v: " << v.getSize() << '\n';
```

```
Vector& r{v}; // Referenz der Variable 'r'  
std::cout << "Größe von r: " << r.getSize();
```

```
Größen von *p: 3, v: 3  
Größe von r: 3
```

```
Vector arr[] = { 0,1,2,3,4,5,6,7,8,9 }; // Impliziter Konstruktor-Aufruf!
```

(Funktioniert nicht, wenn
Keyword explicit gesetzt ist)

```
// Gib die Vektoren aus  
for (auto& vec : arr) // Für jedes vec in arr  
{
```

Automatische Typdeduktion

```
    std::size_t size = vec.getSize();  
    std::cout << "Größe: " << size << ", Elemente: (" ;  
    for (std::size_t i = 0; i < size; ++i)  
        std::cout << vec.at(i) << " ";  
    std::cout << ")\n";  
}
```

```
Größe: 0, Elemente: ( )  
Größe: 1, Elemente: ( 0 )  
Größe: 2, Elemente: ( 0 0 )  
Größe: 3, Elemente: ( 0 0 0 )  
Größe: 4, Elemente: ( 0 0 0 0 )  
Größe: 5, Elemente: ( 0 0 0 0 0 )  
Größe: 6, Elemente: ( 0 0 0 0 0 0 )  
Größe: 7, Elemente: ( 0 0 0 0 0 0 0 )  
Größe: 8, Elemente: ( 0 0 0 0 0 0 0 0 )  
Größe: 9, Elemente: ( 0 0 0 0 0 0 0 0 0 )
```

Anwendung

This-Pointer

- Der **this**-Pointer ist ein spezieller Zeiger, der in jeder (nicht statischen) Methode einer Klasse verfügbar ist
 - Er zeigt auf die aktuelle Instanz der Klasse, von dem die Methode aufgerufen wurde
 - Mit dem `this`-Pointer kann man:
 1. Auf die Daten der aktuellen Instanz zugreifen (z.B. bei Namenskonflikten)
 2. Die Adresse der aktuellen Instanz erhalten, um sie weiterzugeben
- Der `this`-Pointer ist implizit
 - Er muss nicht verwendet werden, um auf die Daten der Klasse zuzugreifen



Person
name: std::string
age: int
getAddress: Person*

Person.cpp

```
#include "Person.h"
// Konstruktor
Person::Person(std::string name, int age) {
    this->name = name;
    this->age = age;
}
// Setter
Person::Person& setName(std::string name) {
    this->name = name; // Explizite Verwendung des this-Pointers
    return *this; // Ermöglicht Method-Chaining
}
Person::Person& setAge(int age) {
    this->age = age;
    return *this;
}
// Gibt die Adresse des aktuellen Objekts zurück
Person::Person* getAddress() {
    return this;
}
```

Die Attribute sind gleich wie die Funktionsparameter benannt. Mithilfe von `this` wird spezifiziert, ob das Attribut oder der Parameter gemeint ist.

Gibt eine Referenz auf die Instanz zurück

Gibt die Adresse des Objekts zurück

Anwendung

New & Delete



- Instanzen können dynamisch zur Laufzeit mithilfe von **new** allokiert werden
 - Zum Freigeben des reservierten Speicherplatzes wird **delete** verwendet
 - Mit der Freigabe des Speichers über `delete` wird der **Destruktor** aufgerufen

```
#include "Person.h"
// Konstruktor
Person::Person(std::string name, int age) {
    this->name = name;
    this->age = age;
    std::cout << "Person erstellt: " << this->name << ", "
              << this->age << " Jahre alt." << '\n';
}
// Destruktor
Person::~~Person() {
    std::cout << "Person gelöscht: " << this->name << '\n';
}
/* ... */
```

Person.cpp

```
#include "Person.h"
int main() {
    // Dynamische Erstellung einer Person
    Person* person{ new Person{"Max", 25} };

    // Verwenden von Method-Chaining
    person->setName("Anna").setAge(30);

    // Adresse der dynamisch erstellten Instanz anzeigen
    std::cout << "Adresse des Objekts: "
              << person->getAddress() << '\n';

    // Speicher freigeben
    delete person; // Destruktor wird aufgerufen
}
```

```
Person erstellt: Max, 25 Jahre alt.
Adresse des Objekts: 0000019A7D408680
Person gelöscht: Anna
```

Anwendung

Dokumentation



- Die **Dokumentation** von Programmcode (insbesondere von Klassen und Funktionen) ist ein wichtiger Teil der Softwareentwicklung
 - Zum einen kann der eigene Code besser lesbar werden,
 - zum anderen wird der Code besser nachvollziehbar für andere Entwickler
- Eine beliebte Vorgehensweise dabei ist, die Programmdokumentation **direkt im Code** – mittels Kommentares (`// Kommentar` oder `/*Kommentar*/`) – zu verfassen
 - Eine Programmdokumentation separat (Word, LaTeX, ...) mit dem Programmcode mitzuführen ist sehr wartungsintensiv und redundant
- Neben einzelnen Codezeilen, wird häufig ein **Kommentarblock** über einer Funktion oder einer Klasse verfasst, der detaillierte Informationen zu diesen enthält
 - Dabei existieren unterschiedliche **Stile**

Anwendung



Dokumentation – Stil: Doxygen

- Der „**Doxygen**“-Stil ist ein weit verbreiteter Stil für Programmdokumentationen über Kommentarblöcke in verschiedenen Programmiersprachen
 - Dabei wird ein bereichsbasierter Kommentar (*/*Kommentar*/*) genutzt und einzelne Eigenschaften über die Notation mit dem @-Symbol definiert
- Beispiel: Doxygen-Stil Dokumentation einer Quadrierungsfunktion

```
/**
 * @brief Berechnet das Quadrat von x
 *
 * @param x Gleitkommazahl, deren Quadrat berechnet werden soll
 * @return Quadrat von x
 */
constexpr float square(float x)
{
    return x * x;
}
```

@brief = kurze (brief) Beschreibung der Funktion oder Klasse

@param x = Beschreibung des Parameters x

@return = Beschreibung des Rückgabewerts

Details zur Notation: <https://www.doxygen.nl/manual/docblocks.html>

Anwendung



Dokumentation – Stil: XML

- Der Standard-Stil für Programmdokumentationen über Kommentarblöcke in Visual Studio basiert auf einer **XML**-Notation
 - Dabei werden mehrere zeilenbasierte Kommentare mit einem zusätzlichen Slash (`/// Kommentar`) genutzt und einzelne Eigenschaften über XML-Tags (`<tag>Eigenschaft</tag>`) definiert
- Beispiel: XML-Stil Dokumentation einer Quadrierungsfunktion

```
/// <summary>Berechnet das Quadrat von x</summary>
/// <param name="x">
/// Gleitkommazahl, deren Quadrat berechnet werden soll
/// </param>
/// <returns>Quadrat von x</returns>
constexpr float square(float x)
{
    return x * x;
}
```

`<summary>` = Beschreibung der Funktion oder Klasse

`<param name="x">` = Beschreibung des Parameters `x`

`<returns>` = Beschreibung des Rückgabewerts

Details zur Notation: <https://learn.microsoft.com/en-us/cpp/build/reference/xml-documentation-visual-cpp>

Anwendung



Dokumentation – Entwicklungsumgebungen

- Moderne Entwicklungsumgebungen (IDEs) verarbeiten die Dokumentation **live** und unterstützen den Programmierer damit **während dem Entwicklungsprozess**

```
square(2);
```

constexpr float square(float x)

Berechnet das Quadrat von x

Parameters:
x Gleitkommazahl, deren Quadrat berechnet werden soll

Returns:
Quadrat von x

[Search Online](#)

Live-Tooltip bei Hover über Funktionsname

```
1223 | float y = square();
```

```
constexpr float square(float x)
```

Berechnet das Quadrat von x

x: Gleitkommazahl, deren Quadrat berechnet werden soll

Live-Tooltip beim Tippen der Funktionsparameter

- Diese Hilfestellungen können den Entwicklungsprozess erheblich beschleunigen

Anwendung

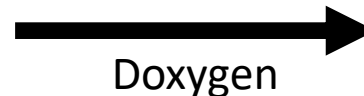
Dokumentation – Generierung



- Aus den Kommentarblöcken kann eine **externe Programmdokumentation** (Webseite, LaTeX, PDF, ...) generiert werden
 - Bekannte Generatoren hierzu sind „Doxygen“ oder „GhostDoc“
- Beispiel: Generierung mit Doxygen

```
/**  
 * @brief Berechnet das Quadrat von x  
 *  
 * @param x Gleitkommazahl, deren Quadrat berechnet werden soll  
 * @return Quadrat von x  
 */  
constexpr float square(float x)  
{  
    return x * x;  
}
```

Dokumentation im Code



Doxygen

Functions

constexpr float **square** (float x)
Berechnet das Quadrat von x. More...

Function Documentation

◆ **square()**

constexpr float square (float x)

Berechnet das Quadrat von x.

Parameters

x Gleitkommazahl, deren Quadrat berechnet werden soll

Returns

Quadrat von x

Generated by **doxygen** 1.9.5

Dokumentation als HTML-Webseite

Anwendung

Zusammenfassung



- Verwende für die **Initialisierung** von Objekten `{ }` oder `()`
 - In den meisten Fällen ist eine Initialisierung mit geschweiften Klammern sinnvoll
 - Ein Konstruktor mit **Initializer-List** wird bei der Initialisierung mit `{ }` bevorzugt
 - Es sollte immer bekannt sein, ob eine **Initialisierung oder Zuweisung** geschieht
- Klassen können wie Standard-Datentypen verwendet werden
 - Pointer, Referenzen, Arrays und Speicherallokationen funktionieren äquivalent
- Der **this**-Pointer ist ein Zeiger auf die aktuelle Instanz der Klasse
- Um Klassen gut zu dokumentieren und leichter an Informationen zur Funktionsweise zu kommen, können **Kommentarblöcke im Doxygen- oder XML-Stil** verwendet werden.



Spezielle Schlüsselwörter

Lernziele

- Das Schlüsselwort **const** und seine Anwendungsfälle wurden verstanden
 - `const` kann korrekt in eigenen Projekten verwendet werden
- Das Schlüsselwort **static** und seine Anwendungsfälle wurden verstanden
- Das Schlüsselwort **inline** und seine Verwendung sind bekannt

Spezielle Schlüsselwörter

Konstante Funktions-Parameter



- Konstanten tragen erheblich dazu bei, die **Lesbarkeit** des Programmcodes zu verbessern und **Schnittstellen** korrekt zu definieren

```
class TextureProcessor
{
public:
    void processTexture(const Texture& texture);
};
```

const garantiert – ohne weitere Dokumentationen – dass das übergeben Textur-Objekt durch die Funktion nicht verändert wird

```
void loadAndProcessTexture(TextureProcessor& tp)
{
    Texture t;
    t.loadFromFile("wand.png");

    tp.processTexture(t);

    ...
}
```

Bei weiterer Verwendung des Textur-Objekts kann davon ausgegangen werden, dass es durch den Aufruf von `processTexture()` **nicht** verändert wurde, **ohne die konkrete Implementierung von `TextureProcessor` zu kennen!**

Spezielle Schlüsselwörter



Konstante Attribute

- Wenn ein Attribut bei der Erzeugung der Instanz gesetzt wird und sich danach nicht mehr Ändern können soll, spricht man von einem **konstanten Attribut**
 - Konstante Attribute können ausschließlich bei der **Doppelpunkt-Initialisierung** des Konstruktors initialisiert werden

```
class Vector {  
public:  
    explicit Vector(std::size_t size);  
  
    void setSize(std::size_t size);  
    std::size_t getSize();  
    double& at(std::size_t index);  
  
private:  
    double* m_elements;  
    // Anzahl der Elemente ist konstant  
    const std::size_t m_size;  
};
```

Vector.h

```
#include "Vector.h"  
Vector::Vector(std::size_t size)  
    : m_elements{new double[size]}  
    , m_size {size}  
{  
    for (std::size_t i=0; i < m_size; ++i)  
        m_elements[i] = 0;  
    m_size = size; // FEHLER!  
}  
  
// Für konstante Attribute ergeben Setter keinen Sinn!  
void Vector::setSize(std::size_t size) {  
    m_size = size; // FEHLER!  
};
```

Vector.cpp

Konstante Attribute müssen hier initialisiert werden!

m_size kann nicht mehr verändert werden.

Spezielle Schlüsselwörter



Konstante Methoden

- Auf konstanten Objekten können nur **konstante Methoden** aufgerufen werden
- Konstante Methoden werden durch Anhängen des Keywords **const** ans **Ende der Funktionssignatur** deklariert
 - Dies schaltet den **this-Pointer** konstant und damit alle Membervariablen und -methoden des Objekts

```
class Texture {  
public:  
    void printData() const {  
        // Keine Veränderung bei Ausgabe  
        std::cout << "Pixels: " << (m_pixels ? "exist" : "don't exist")  
            << ", Count: " << m_pixelCount;  
        m_pixelCount = 5; // Compilerfehler  
    }  
private:  
    uint8_t* m_pixels;  
    size_t m_pixelCount;  
};
```

const am Ende der Funktionssignatur kennzeichnet, dass die Funktion das eigene Objekt nicht verändern wird

Membervariablen werden im Kontext von konstanten Methoden zu Konstanten und können nicht verändert werden

Spezielle Schlüsselwörter

Konstante Methoden – Getter



- **Getter** werden immer als **konstante Methoden** deklariert
 - Durch den Aufruf eines Getters dürfen keine Attribute verändert werden

```
class Vector {  
public:  
    explicit Vector(std::size_t size);  
  
    void setSize(std::size_t size);  
    std::size_t getSize() const;  
    double& at(std::size_t index);  
  
private:  
    double* m_elements;  
    // Anzahl der Elemente ist konstant  
    const std::size_t m_size;  
};
```

Vector.h

```
/* ... */  
std::size_t Vector::getSize() const {  
    return m_size;  
}
```

Vector.cpp

Spezielle Schlüsselwörter

Konstante Methoden – mutable



- In **Ausnahmefällen** muss eine Membervariable innerhalb einer konstanten Methode trotzdem verändert werden können
 - Z.B. beim Caching, Lazy-Loading oder bei einer Mutex im Multithreading
- Dazu können Membervariablen **mutable** (= veränderlich) deklariert werden
 - Dies hat nur Auswirkungen im Kontext konstanter Methoden
- mutable Membervariablen sind ein „Hack“ und sollten möglichst nicht verwendet werden

Beispiel: Lazy-Loading bzw. Caching

```
class TextureFactory {  
public:  
    std::shared_ptr<Texture> getOrCreateWallTexture() const {  
        std::shared_ptr<Texture> textureWall = m_textureWall.lock();  
        if (textureWall == nullptr) {  
            textureWall = std::make_shared<Texture>();  
            textureWall->loadFromFile("wall.png");  
  
            m_textureWall = textureWall;  
        }  
        return textureWall;  
    }  
private:  
    mutable std::weak_ptr<Texture> m_textureWall;  
};
```

std::shared_ptr werden in einer späteren Vorlesung erklärt

m_textureWall ist trotz konstanter Methode zuweisbar, durch das mutable Keyword

Spezielle Schlüsselwörter

Static – Übersicht



- Unter „**statisch**“ versteht man programmiersprachenübergreifend die **einmalige Existenz** eines Typs, unabhängig von Instanzen
 - Das Keyword **static** hat in C++ unterschiedliche Funktionen
 - Bei einem **Klassen-Member** (Methode oder Attribut)
 - Der Member existiert unabhängig von Instanzen und nur einmal für die gesamte Klasse
 - Bei einem **globalen Typ** (Funktion oder Variable)
 - Der Typ existiert nur für die aktuelle Compiler-Einheit
 - keine Transitivität („durchsickern“) in andere Compiler-Einheiten (= „internal linkage“)
- ```
// statische globale Variable
static float g_buttonWidth = 80.0f;
```
- Bei einer **lokalen Variable** (= in einer Funktion)
    - Die Variable existiert über Funktionsaufrufe hinweg und nur einmal
    - Sie ist nur im Scope der Funktion sichtbar
    - Die Variable wird beim ersten Funktionsaufruf erzeugt

# Spezielle Schlüsselwörter

## Statische Klassen-Member



- Ein **statischer Klassen-Member** existiert unabhängig von den Instanzen
  - Es benötigt keine Instanz, um auf den Member zugreifen zu können
  - Der Member existiert einmal für die gesamte Klasse
  - Alle Instanzen greifen auf denselben Member zu
- **Statische Attribute** müssen außerhalb des Klassen-Körpers definiert werden

```
int main() {
 std::cout << MyClass::getInstanceCount() << '\n';
 MyClass i0, i1, i2;
 std::cout << i0.getInstanceCount() << '\n';
 {
 MyClass i3;
 std::cout << MyClass::getInstanceCount() << '\n';
 }
 std::cout << i2.getInstanceCount() << '\n';
}
```

0  
3  
4  
3

```
class MyClass {
public:
 MyClass();
 ~MyClass();
 // statische Methode
 static int getInstanceCount();
private:
 // statische Variable
 static int ms_instanceCount;
};
```

MyClass.h

```
#include "MyClass.h"
MyClass::MyClass() { ++ms_instanceCount; }
MyClass::~~MyClass() { --ms_instanceCount; }
int MyClass::getInstanceCount() {
 return ms_instanceCount;
}
// statische Variable wird definiert
int MyClass::ms_instanceCount = 0;
```

MyClass.cpp

# Spezielle Schlüsselwörter



## Statische lokale Variable

- Eine **statische lokale Variable** (= in einer Funktion) existiert einmalig über alle Funktionsaufrufe hinweg
  - Die Variable ist nur im Scope der Funktion sichtbar
  - Beim ersten Funktionsaufruf wird die Variable einmalig erzeugt
  - Beim Verlassen der Funktion wird die Variable **nicht** gelöscht
  - Beim erneuten Funktionsaufruf wird auf die bereits angelegte Variable zugegriffen

```
void printCallCount()
{
 // statische lokale Variable
 static int callCount = 0;
 std::cout << ++callCount << ", ";
}
```

1,2,3,4,5,6,7,8,9

```
int main()
{
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
 printCallCount();
}
```

# Spezielle Schlüsselwörter



## Inline-Substitution

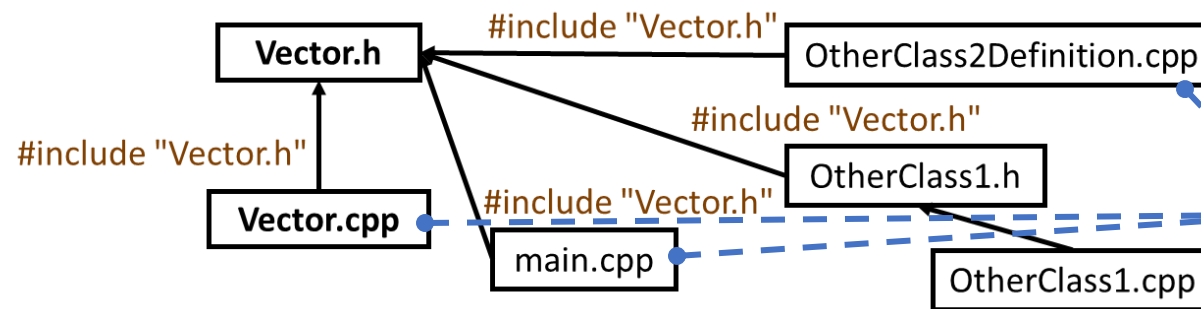
- Die ursprüngliche Absicht des **inline** Schlüsselwortes war, Funktionen zu kennzeichnen, die der Compiler ohne Funktionsaufruf in den Code einfügen darf
  - Wird als **Inline-Substitution** bezeichnet
  - Setzt den Code der Funktion an den Stellen in den Code ein, wo sie aufgerufen wird
  - Führt eben keinen Funktionsaufruf mehr durch
  - Vermeidet Overhead, der ansonsten durch den Funktionsaufruf geschieht
  - Die Executable-Datei kann dadurch jedoch deutlich größer werden
- Inline-Substitution in der C++ Semantik ist nicht beobachtbar
  - Der Compiler kann Inline-Substitution für beliebige Funktionen verwenden, auch, wenn sie nicht als `inline` gekennzeichnet sind
  - Der Compiler darf auch bei Funktionen, die als `inline` gekennzeichnet sind, Funktionsaufrufe erzeugen
- **Verwechsle das `inline` Schlüsselwort NICHT mit Inline-Substitution!**

# Spezielle Schlüsselwörter

## Inline



- **Inline** kennzeichnet Funktionen, für die mehrere Definitionen erlaubt sind
  - Die Definitionen müssen in allen Compiler-Einheiten übereinstimmen
  - Funktionen, die komplett im Klassen-Körper definiert sind, sind implizit `inline`
  - Meistens ist es geschickter, die Definition und Deklaration sauber zu trennen, statt `inline` zu verwenden



```
class Vector {
public:
 Vector()
 : m_size{0}
 , m_elements{nullptr}
 {}
 std::size_t getSize() const;
 /* ... */
};
inline std::size_t Vector::getSize() const {
 return m_size;
}
```

Vector.h

Durch die Definition im Klassenkörper ist der Konstruktor **implizit inline** deklariert.

Die Definition der Inline-Funktionen existiert **mehrfach**:  
In jeder Quellcode-Datei, die Vector.h inkludiert

# Spezielle Schlüsselwörter



## Inline

- **C++17** **inline** kennzeichnet auch **Variablen** mit einer statischen Datenspeicherung (z.B. statische Attribute oder Namespace-Variablen), deren Initialisierung für mehrere Compiler-Einheiten sichtbar ist.

```
class MyClass {
public:
 MyClass();
 ~MyClass();
 // statische Methode
 static int getInstanceCount();
private:
 // statische Variable
 inline static int ms_instanceCount = 0;
};
```

MyClass.h

Durch `inline` kann das statische Attribut nun innerhalb des Klassenkörpers definiert werden.

# Spezielle Schlüsselwörter

## Zusammenfassung



- **Konstanten** tragen erheblich dazu bei, die Lesbarkeit des Programmcodes zu verbessern und Schnittstellen korrekt zu definieren
  - Stellen sicher, dass an Funktionen übergebene Objekte nicht verändert werden
  - Stellen sicher, dass Attribute nach der Initialisierung nicht mehr verändert werden
  - Stellen sicher, dass Methoden keine Attribute verändern
    - Außer bei Attributen, die mit **mutable** gekennzeichnet sind
- Unter „**statisch**“ versteht man programmiersprachenübergreifend die **einmalige Existenz** eines Typs, unabhängig von Instanzen
  - Bei einem **Klassen-Member** existiert er unabhängig von Instanzen und nur einmal für die gesamte Klasse
  - Bei einem **globalen Typ** existiert er nur für die aktuelle Compiler-Einheit
  - Bei einer **lokalen Variable** existiert sie über Funktionsaufrufe hinweg und nur einmal
- **Inline** kennzeichnet Funktionen und Variablen, deren Definition in mehreren Compiler-Einheiten eingebunden werden



# Spezielle Schlüsselwörter

## Verständnisfragen

PIN: 569316

<https://poll.hs-kempten.de/poll/>



- **Wofür nutzt man das Schlüsselwort `static` in C++?**
  - Um eine Funktion nur für Instanzen einer Klasse verfügbar zu machen.
  - Für Variablen, die im gesamten Programmablauf nur einmalig existieren.
  - Für lokale Variablen in Funktionen, um ihren Wert zwischen Aufrufen beizubehalten.
  - Für Methoden, die aufgerufen werden können, ohne eine Instanz der Klasse zu erstellen.
- **WAHR oder FALSCH?**
  - Mit dem Schlüsselwort **`const`** deklarierte Member-Variablen können nach der Initialisierung nur durch Member-Funktionen geändert werden, die ebenfalls als **`const`** deklariert sind.
  - Ein Konstruktor einer Klasse kann als **`const`** deklariert werden.
  - Ein **`const`**-Objekt einer Klasse kann nur **`const`**-Methoden aufrufen.
  - Eine Klassenmethode, die als **`const`** deklariert ist, kann auf alle Member-Variablen der Klasse zugreifen.
  - Ein **`const`**-Member einer Klasse muss immer bei der Definition der Klasse initialisiert werden.
  - Das Schlüsselwort **`static`** kann in C++ nur auf Membervariablen angewendet werden.
  - **`inline`** gibt an, bei welchen Funktionen der Compiler den Funktionsaufruf direkt mit dem Quellcode ersetzen soll.
  - **`inline`** kennzeichnet Funktionen und Variablen, deren Definition von mehreren Quellcodedateien eingebunden werden darf.



# Überladung

## Lernziele

- Die Funktionsweise von Methoden-Überladung wurde verstanden
  - Es kann abgeschätzt werden, welche Methode vom Compiler aufgerufen wird
- Für eigene Klassen können die Operatoren überladen werden
  - Es ist bekannt, wann Operatoren innerhalb und wann außerhalb des Klassen-Scopes überladen werden

# Überladung

## Methoden-Überladung



- Genau wie Funktionen können auch **Methoden überladen** werden
  - Eine Klasse kann mehrere Methoden mit demselben Namen, aber unterschiedlichen Parametern zur Verfügung stellen
  - Beim Aufruf der Methode entscheidet der Compiler, welche Überladung am geeignetsten ist
  - Eine nicht-konstante Methode kann mit einer konstanten Methode überladen werden

```
class Calculator {
 public:
 int add(int a, int b);
 int add(int a, int b, int c);
 double add(double a, double b);
 double add(int a, double b);
};
```

```
5 + 7 = (Add Version 1) 12
3 + 4 + 5 = (Add Version 2) 12
2.5 + 3.8 = (Add Version 3) 6.3
10 + 4.5 = (Add Version 4) 14.5
```

```
#include "Calculator.h"
int Calculator::add(int a, int b) {
 std::cout << "(Add Version 1) "; return a + b;
}
int Calculator::add(int a, int b, int c) {
 std::cout << "(Add Version 2) "; return a + b + c;
}
double Calculator::add(double a, double b) {
 std::cout << "(Add Version 3) "; return a + b;
}
double Calculator::add(int a, double b) {
 std::cout << "(Add Version 4) "; return a + b;
}
```

```
#include "Calculator.h"
int main() {
 Calculator calc;
 std::cout << "5 + 7 = "
 << calc.add(5, 7) << '\n';
 std::cout << "3 + 4 + 5 = "
 << calc.add(3, 4, 5) << '\n';
 std::cout << "2.5 + 3.8 = "
 << calc.add(2.5, 3.8) << '\n';
 std::cout << "10 + 4.5 = "
 << calc.add(10, 4.5) << '\n';
}
```

# Überladung

## Methoden-Überladung



- Wir haben Methoden-Überladungen bereits bei **Konstruktoren** kennengelernt

```
class Vector {
public:
 /// <summary>
 /// Standard-Konstruktor.
 /// </summary>
 Vector();
 /// <summary>
 /// Konstruktor mit gegebener Größe.
 /// Alle Elemente werden auf den Standardwert 0.0 gesetzt.
 /// </summary>
 /// <param name="size">Die Anzahl der Elemente des Vektors.</param>
 explicit Vector(std::size_t size);
 /// <summary>
 /// Konstruktor mit gegebener Größe und Initialwert.
 /// </summary>
 /// <param name="size">Die Anzahl der Elemente des Vektors.</param>
 /// <param name="initialValue">Der Initialwert jedes Elements.</param>
 Vector(std::size_t size, double initialValue);
 /* ... */
};
```

```
#include "Vector.h"
int main() {
 Vector vec1;
 std::cout << "Größe von vec1: "
 << vec1.getSize() << '\n'; Größe von vec1: 0

 Vector vec2(5);
 std::cout << "Größe von vec2: "
 << vec2.getSize() << '\n'; Größe von vec2: 5

 Vector vec3(3, 2.5);
 std::cout << "Größe von vec3: "
 << vec3.getSize() << '\n'; Größe von vec3: 3

 std::cout << "Elemente von vec3: ";
 for (std::size_t i = 0; i < vec3.getSize(); ++i) {
 std::cout << vec3.at(i) << ' ';
 } Elemente von vec3: 2.5 2.5 2.5
 std::cout << '\n';
}
```

# Überladung



## Operator-Überladung

- Damit mit den eigenen Klassen genauso gearbeitet werden kann wie mit Basis-Datentypen, kann durch das **Überladen von Operatoren** das Verhalten der Klasse für den jeweiligen Operator definiert werden.

```
#include "Vector.h"
```

```
int main() {
```

```
 Vector v1(3);
```

```
 v1[0] = 2.5;
```

```
 v1[1] = 1.4;
```

```
 v1[2] = 3.14;
```

Zugriffsoperator []

```
 Vector v2(4);
```

```
 std::cin >> v2;
```

Eingabeoperator >>

```
 std::cout << "Vector 1: " << v1 << '\n';
```

Ausgabeoperator <<

```
 if (v1 != v2)
```

Vergleichsoperator !=

```
 std::cout << "Vector 2: " << ++v2;
```

Prefix-Inkrement ++

```
}
```

*gewünschte Ausgabe:*

Index 0: 5.3

Index 1: 1.6

Index 2: 7.2

Index 3: 8.9

Vector 1: ( 2.5 , 1.4 , 3.14 )

Vector 2: ( 6.3 , 2.6 , 8.2 , 9.9 )

# Überladung

## Operator-Überladung



- Diese Operatoren können überladen werden:
  - Binäre arithmetische Operatoren: `+`, `-`, `*`, `/` und `%`
  - Binäre logische Operatoren: `&` (Bitwise AND), `|` (Bitwise OR) und `^` (Bitwise XOR)
  - Binäre Vergleichsoperatoren: `==`, `!=`, `<`, `<=`, `>`, `>=` und `<=>`
  - Unäre arithmetische und logische Operatoren: `+`, `-`, `~` und `!`
  - Zuweisungsoperatoren: `=`, `+=`, `*=`, ...
  - Inkrement und Dekrement: `++` und `--`
  - Pointer Operationen: `->`, unäres `*` und unäres `&`
  - Aufrufoperator: `()`
  - Indizierung: `[]`
  - Logische Operatoren: `&&` und `||`
  - Komma: `,`
  - Verschiebeoperatoren: `>>` und `<<`

Es gibt nie gute Gründe, um `&&`, `||` oder `,` zu überladen!

# Überladung

## Operator-Überladung



- Operatoren können als **Methode** definiert werden
  - Die eigene Instanz muss nicht als Parameter übergeben werden
  - Wird für Operatoren genutzt, die das erste Element der Operation verändern (z.B. +=)
  - Ist verpflichtend für =, ->, () und []
- Alternativ ist die Definition als **freistehende Funktion** möglich
  - Empfiehlt sich besonders für Operatoren mit symmetrischen Operanden (also bei Operationen, bei denen die Reihenfolge der Operanden keinen Unterschied macht)
  - Ist verpflichtend für die Aus- und Eingabe ( >> und << )

```
class Vector {
public:
 /* ... */
 double& operator[] (std::size_t idx);
}
```

Operator-Überladung  
innerhalb der Klasse.

Vector.h

Operator-Überladung  
außerhalb der Klasse.

```
std::ostream& operator<< (std::ostream& os, const Vector& v);
```

Die Instanz muss übergeben werden.

```
int main() {
 // Vector mit 6 Elementen
 Vector v1{6};
 // Setzt das 0. Element auf 2.5
 v1[0] = 2.5;
 // Gib den Vector aus
 std::cout << v1 << '\n';
}
```

# Überladung

## Indizierungsoperator



- Um über die eckigen Klammern `[]` auf in der Klasse verwaltete Elemente zugreifen zu können, muss der **Indizierungsoperator** überladen werden
  - Der Indizierungsoperator muss zwingend im Klassen-Körper definiert werden

```
class Vector {
public:
 // Element darf verändert werden
 double& operator[](std::size_t idx);
 // Element darf nicht verändert werden
 double operator[](std::size_t idx) const;
}
```

Vector.h

Der Zugriffsoperator  
ist konstant und nicht-  
konstant überladen.

```
double& Vector::operator[](std::size_t idx) {
 // Wenn idx >= m_size, gib das letzte Element zurück
 return m_elements[std::min(m_size - 1, idx)];
}

double Vector::operator[](std::size_t idx) const {
 return m_elements[std::min(m_size - 1, idx)];
}
```

Vector.cpp

```
#include "Vector.h"
```

```
int main() {
```

```
 Vector v1(3);
```

```
 v1[0] = 2.5;
```

```
 v1[1] = 1.4;
```

```
 v1[2] = 3.14;
```

```
 const Vector& r(v1);
```

```
 std::cout << r[0] << ' ';
```

```
 << r[1] << ' ';
```

```
 << r[2] << '\n';
```

```
}
```

2.5,1.4,3.14



# Überladung

## Vergleichsoperatoren



- Überladung von **Vergleichsoperatoren**, um eigene Objekte zu vergleichen
  - Können beliebig implementiert werden, sollten aber den **Erwartungen des Nutzers** entsprechen
  - Best Practice: Wenn `==` definiert ist: `!=` als `!(a==b)` definieren
  - Der Wahrheitswert von `a<=b` sollte derselbe sein wie `(a<b) || (a==b)` und wie `!(b>a)`
  - `a>b` sollte gleichbedeutend mit `b<a` sein

### In der Klasse / Member-Funktion

```
class Vector {
 public:
 bool operator==(const Vector& other) const;
}
```

Vector.h

```
#include "Vector.h"
bool Vector::operator==(const Vector& other) const {
 /* ... */
}
```

Vector.cpp

### Außerhalb der Klasse / freistehende Funktion

```
class Vector {
 /* ... */
}
bool operator==(const Vector& v1, const Vector& v2);
```

Vector.h

```
#include "Vector.h"
bool operator==(const Vector& v1, const Vector& v2){
 /* ... */
}
```

Vector.cpp

# Überladung

## Vergleichsoperatoren



- Die Definition als **freistehende Funktion** ermöglicht, je nach Kontext unterschiedliche Vergleichsfunktionen bereitzustellen

```
bool operator==(const Vector& lhs, const Vector& rhs) {
 // Anzahl der Elemente muss übereinstimmen
 if (lhs.getSize() != rhs.getSize())
 return false;
 // WICHTIG: operator[] muss als const Überladung existieren
 for (std::size_t i = 0; i < lhs.getSize(); ++i)
 if (lhs[i] != rhs[i])
 return false;
 return true;
}
```

Bei dem Vergleich von Objekten gibt es nie einen Grund, die Objekte zu verändern!

Ruft den `operator[] const` auf, da die Instanzen als konstante Referenzen übergeben werden.

```
bool operator!=(const Vector& lhs, const Vector& rhs) {
 // Ruft den operator== auf und invertiert das Ergebnis
 return !(lhs == rhs);
}
```

In einer Operator-Überladung dürfen beliebige weitere Operator-Überladungen verwendet werden.

# Überladung

## Vergleichsoperatoren



- **C++20** Der **Spaceship Operator** `<=>`
  - Gibt nicht *True* oder *False* zurück
  - `a <=> b` → negativ: a ist kleiner als b
  - `a <=> b` → 0: a und b sind gleich
  - `a <=> b` → positiv: a ist größer als b
  - Definiert implizit alle Vergleichsoperatoren außer `==` und `!=`

```
void compare(const Vector& v1, const Vector& v2)
{
 bool b1{ (v1 <=> v2) == 0 }; // v1 == v2
 bool b2{ (v1 <=> v2) < 0 }; // v1 < v2
 bool b3{ (v1 <=> v2) > 0 }; // v1 > v2
 bool b4{ (v1 == v2) }; // ERROR: == nicht definiert
 bool b5{ (v1 < v2) }; // OK
}
```

```
auto operator<=>(const Vector& v1, const Vector& v2)
{
 auto minSize = std::min(v1.getSize(), v2.getSize());

 for (std::size_t i = 0; i < minSize; ++i)
 if (v1[i] != v2[i])
 return v1[i] < v2[i] ? -1 : 1;

 if (v1.getSize() == v2.getSize())
 return 0;
 return v1.getSize() < v2.getSize() ? -1 : 1;
}
```

# Überladung



## Prefix & Postfix Inkrement & Dekrement

- **Prefix Inkrement und Dekrement**  
geben immer eine Referenz auf das veränderte Objekt zurück
- **Postfix Inkrement und Dekrement**  
werden durch ein künstliches Funktionsargument gekennzeichnet
  - Das Objekt muss kopiert werden, bevor sein Wert verändert werden darf
  - Zur Veränderung der Werte wird meistens dann der Prefix Operator aufgerufen
  - Danach muss die Kopie mit dem alten Wert zurückgegeben werden

```
// prefix increment
Vector& operator++(){
 for (std::size_t i = 0; i < m_size; ++i)
 ++m_elements[i];
 // Gib neuen Wert per Referenz zurück
 return *this;
}

// postfix increment
Vector operator++(int){
 // Kopier den alten Wert
 Vector old = *this;
 // Inkrementiere die Elemente
 operator++();
 // Gib den alten Wert zurück
 return old;
}
```

```
Vector v1(2);
v1[0] = 1.3;
v1[1] = -0.9;
++v1; // prefix
std::cout << v1[0];
std::cout << ' ' << v1[1];
// 2.3, 0.1

v1++; // postfix
```

[https://en.cppreference.com/w/cpp/language/operator\\_incredec](https://en.cppreference.com/w/cpp/language/operator_incredec)

# Überladung

## Ausgabe & Eingabe



### ■ Output Operator

- Anwendung des left-shift Operators << im Kontext von *iostreams*
- Definiert, wie ein Objekt zum Beispiel auf der Konsole ausgegeben werden soll
- Erhält eine Referenz auf einen Output-Stream als Parameter und gibt die Referenz als Return Wert zurück

### ■ Input Operator

- Anwendung des right-shift Operators >> im Kontext von *iostreams*
- Definiert wie Nutzereingaben das Objekt verändern können
- Erhält eine Referenz auf einen Input-Stream als Parameter und gibt diese als Return Wert zurück

```
std::ostream& operator<<(std::ostream& os, const Vector& v) {
 auto size = v.getSize();
 if (size == 0)
 return os << "()";

 os << "(";
 for (std::size_t i = 0; i < v.getSize() - 1; ++i)
 os << v[i] << " , ";
 return os << v[size - 1] << ")";
}
```

```
std::istream& operator>>(std::istream& is, Vector& v) {
 for (std::size_t i = 0; i < v.getSize(); ++i) {
 printf("Index %u: ", i);
 is >> v[i];
 }
 return is;
}
```

# Überladung

## friend



- Das **friend** Schlüsselwort innerhalb des Klassen-Körpers ermöglicht einer Funktion oder anderen Klasse den **Zugriff auf nicht-public Member** der Klasse
  - Ein unbedachtes Verwenden von **friend** kann die Datenkapselung zerstören
  - Wird häufig verwendet, wenn eine große Klasse in zwei kleinere Klassen aufgeteilt wird
    - Z.B. weil die Teile unterschiedliche Lebenszeiten oder Anzahl an Instanzen besitzen sollen
    - Diese Teile brauchen allerdings weiterhin vollen Zugriff aufeinander
    - Die Daten bleiben weiterhin innerhalb des Klassen-Kontext gekapselt

```
class BankAccount {
private:
 std::string ownerName; // Name des Kontoinhabers.
 double balance; // Kontostand.

 // Erlaubt der BankManager-Klasse
 // den Zugriff auf private Daten
 friend class BankManager;
}
```

Nur die Klasse BankManager kann von außerhalb des BankAccount auf seine privaten Member zugreifen.

```
class BankManager {
public:
 void deposit(BankAccount& account, double amount) {
 account.balance += amount;
 std::cout << "Einzahlung: " << amount
 << " auf das Konto von "
 << account.ownerName << '\n';
 }
 double checkBalance(const BankAccount& account) const {
 return account.balance;
 }
};
```

# Überladung

## friend



- Besonders bei Operator-Überladungen können **friend** Funktionen notwendig sein
  - Ermöglichen den Zugriff auf Klassen-Member, die ansonsten außerhalb der Klasse nicht sichtbar sein sollen
  - Ermöglichen, Funktionen innerhalb des Klassen-Körpers zu deklarieren, die normalerweise außerhalb deklariert werden müssen → Die Funktionen gelten weiterhin nicht als Member

```
class BankAccount {
private:
 std::string ownerName;
 double balance;
 friend class BankManager;
public:
 // Freund-Funktion für den '+'-Operator
 friend double operator+
 (double value, const BankAccount& acc1);
 // Freund-Funktion für den '<<'-Operator
 friend std::ostream& operator<<
 (std::ostream& os, const BankAccount& account);
};
```

```
#include "BankAccount.h"
double operator+
 (double value, const BankAccount& acc1)
{
 return value + acc1.balance;
}
std::ostream& operator<<
 (std::ostream& os, const BankAccount& account)
{
 os << "Kontoinhaber: " << account.ownerName
 << ", Guthaben: " << account.balance;
 return os;
}
```

Friend-Funktionen sind keine Member!  
→ Auf BankAccount:: muss verzichtet werden

# Überladung

## Zusammenfassung



- Damit die eigenen Klassen **wie primitive Datentypen** verwendet werden können, lassen sich die meisten C++ **Operatoren überladen**.
  - Eine Operatorüberladung ist wie eine Funktion aufgebaut.
  - Die Funktionalität der Überladung sollte der allgemeinen Bedeutung des Operators entsprechen.
- Bevorzuge nicht-Member-, nicht-Friend- Funktionen gegenüber Methoden
  - Erhöht die Kapselung, Flexibilität und Erweiterbarkeit der Funktionalität
  - Verwende **friend**, um den Zugriff auf alle Member nur für bestimmte Klassen/Funktion zu ermöglichen



# Überladung

## Verständnisfragen

**PIN: 569316**

<https://poll.hs-kempten.de/poll/>



### ■ Welche Operatoren können überladen werden?

- ++ und --
- >> und <<
- ()
- %

### ■ WAHR oder FALSCH?

- Einige Operator-Überladungen kann als globale Funktion oder als Member-Funktion implementiert werden.
- Die Zuweisungsoperatoren (=) müssen immer als Member-Funktion einer Klasse überladen werden.
- Wenn der == Operator für den Vergleich zweier Objekte TRUE zurückgibt, gibt der != Operator für dieselben Objekte immer FALSE zurück.
- Durch die Überladung des Spaceship Operators werden implizit alle Vergleichsoperatoren außer == und != definiert.
- Ein überladener +-Operator muss immer symmetrisch sein, d. h. a + b und b + a müssen dasselbe Ergebnis liefern.